In [1]:
```
%pip install openpyxl
```

```
Requirement already satisfied: openpyxl in c:\users\anyab\appdata\local\package
s\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\
python311\site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\anyab\appdata\local\packa
ges\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-package
s\python311\site-packages (from openpyxl) (2.0.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import pandas as pd
import numpy as np
import time
from collections import defaultdict, Counter
from itertools import combinations

df = pd.read_excel('Online Retail.xlsx')
print(f"Размер данных: {df.shape}")
```

```
Размер данных: (541909, 8)
```

In [27]:
```
def prepare_transactions(df):
    df = df[~df['InvoiceNo'].astype(str).str.startswith('C')]
    df = df[df['Quantity'] > 0]
    df = df[df['UnitPrice'] > 0]
    df = df[df['Description'].notna()]

    transactions = df.groupby('InvoiceNo')['Description'].apply(list).tolist()

    print(f"Количество транзакций: {len(transactions)}")
    print(f"Уникальных товаров: {len(set([item for t in transactions for item

    return transactions

transactions = prepare_transactions(df)

SAMPLE_SIZE = 1000
transactions_sample = transactions[:SAMPLE_SIZE]
```

```
Количество транзакций: 19960
Уникальных товаров: 4026
```

In [28]:
```
class Apriori:
    def __init__(self, min_support=0.01, min_confidence=0.3, min_lift=1.0):
        self.min_support = min_support
        self.min_confidence = min_confidence
        self.min_lift = min_lift
        self.rules = []

    def _get_support(self, itemset, transactions):
```

```
        count = 0
        itemset_set = set(itemset)
        for trans in transactions:
            if itemset_set.issubset(set(trans)):
                count += 1
        return count / len(transactions)

    def _generate_candidates(self, prev_frequent, k):
        candidates = []
        for i in range(len(prev_frequent)):
            for j in range(i + 1, len(prev_frequent)):
                candidate = sorted(set(prev_frequent[i]) | set(prev_frequent[j
                if len(candidate) == k:
                    all_subsets = True
                    for subset in combinations(candidate, k-1):
                        if list(subset) not in prev_frequent:
                            all_subsets = False
                            break
                    if all_subsets and candidate not in candidates:
                        candidates.append(candidate)
        return candidates

    def fit(self, transactions):
        self.transactions = transactions
        n = len(transactions)
        min_count = self.min_support * n

        counts = Counter()
        for trans in transactions:
            for item in set(trans):
                counts[item] += 1

        frequent = [[item] for item, cnt in counts.items() if cnt >= min_count
        frequent.sort()
        all_frequent = [frequent]

        k = 2
        while all_frequent[-1]:
            candidates = self._generate_candidates(all_frequent[-1], k)
            if not candidates:
                break

            supp_counts = defaultdict(int)
            for trans in transactions:
                trans_set = set(trans)
                for cand in candidates:
                    if set(cand).issubset(trans_set):
                        supp_counts[tuple(cand)] += 1

            frequent_k = []
            for cand in candidates:
                if supp_counts[tuple(cand)] >= min_count:
                    frequent_k.append(cand)
```

```
            if frequent_k:
                all_frequent.append(frequent_k)
            k += 1

        self.rules = []
        for freq_set in all_frequent:
            for itemset in freq_set:
                if len(itemset) < 2:
                    continue
                for i in range(1, len(itemset)):
                    for ante in combinations(itemset, i):
                        ante = list(ante)
                        conseq = [item for item in itemset if item not in ante

                        supp_ante = self._get_support(ante, transactions)
                        supp_conseq = self._get_support(conseq, transactions)
                        supp_union = self._get_support(itemset, transactions)

                        conf = supp_union / supp_ante if supp_ante > 0 else 0
                        lift = supp_union / (supp_ante * supp_conseq) if supp_

                        if conf >= self.min_confidence and lift >= self.min_li
                            self.rules.append({
                                'antecedent': ante,
                                'consequent': conseq,
                                'confidence': conf,
                                'lift': lift
                            })

        self.rules.sort(key=lambda x: x['lift'], reverse=True)
        print(f"Правил: {len(self.rules)}")

    def recommend(self, basket, top_n=5):
        basket_set = set(basket)
        recs = {}
        for rule in self.rules:
            if set(rule['antecedent']).issubset(basket_set):
                for item in rule['consequent']:
                    if item not in basket_set:
                        if item not in recs or rule['lift'] > recs[item]['lift
                            recs[item] = {
                                'item': item,
                                'confidence': rule['confidence'],
                                'lift': rule['lift']
                            }
        result = sorted(recs.values(), key=lambda x: x['lift'], reverse=True)
        return result[:top_n]
```

In [29]:

```
class FPNode:
    def __init__(self, item, count=1, parent=None):
        self.item = item
        self.count = count
```

```
        self.parent = parent
        self.children = {}
        self.next = None

class FPGrowth:
    def __init__(self, min_support=0.01, min_confidence=0.3, min_lift=1.0):
        self.min_support = min_support
        self.min_confidence = min_confidence
        self.min_lift = min_lift
        self.rules = []

    def _get_support(self, itemset, transactions):
        count = 0
        itemset_set = set(itemset)
        for trans in transactions:
            if itemset_set.issubset(set(trans)):
                count += 1
        return count / len(transactions)

    def _build_tree(self, transactions, min_count):
        counts = Counter()
        for trans in transactions:
            for item in set(trans):
                counts[item] += 1

        frequent_items = {item for item, cnt in counts.items() if cnt >= min_c
        if not frequent_items:
            return None, {}

        item_order = sorted(frequent_items, key=lambda x: counts[x], reverse=T
        item_rank = {item: i for i, item in enumerate(item_order)}

        root = FPNode(None)
        header = {item: None for item in frequent_items}

        for trans in transactions:
            filtered = [item for item in trans if item in frequent_items]
            if not filtered:
                continue
            filtered.sort(key=lambda x: item_rank[x])

            current = root
            for item in filtered:
                if item in current.children:
                    current.children[item].count += 1
                else:
                    new_node = FPNode(item, 1, current)
                    current.children[item] = new_node

                    if header[item] is None:
                        header[item] = new_node
                    else:
                        node = header[item]
```

```
                    while node.next:
                        node = node.next
                    node.next = new_node
                current = current.children[item]

        return root, header

    def _mine(self, root, header, min_count, suffix, frequent):
        for item in sorted(header.keys()):
            new_suffix = suffix + [item]
            frequent.append(new_suffix)

            conditional = []
            node = header[item]
            while node:
                path = []
                current = node.parent
                while current and current.item:
                    path.append(current.item)
                    current = current.parent
                if path:
                    for _ in range(node.count):
                        conditional.append(path)
                node = node.next

            if conditional:
                cond_root, cond_header = self._build_tree(conditional, min_cou
                if cond_header:
                    self._mine(cond_root, cond_header, min_count, new_suffix,

    def fit(self, transactions):
        self.transactions = transactions
        n = len(transactions)
        min_count = self.min_support * n

        root, header = self._build_tree(transactions, min_count)
        if root is None:
            print("Нет частых наборов")
            return

        frequent_itemsets = []
        self._mine(root, header, min_count, [], frequent_itemsets)

        self.rules = []
        for itemset in frequent_itemsets:
            if len(itemset) < 2:
                continue
            for i in range(1, len(itemset)):
                for ante in combinations(itemset, i):
                    ante = list(ante)
                    conseq = [item for item in itemset if item not in ante]

                    supp_ante = self._get_support(ante, transactions)
```

```
                    supp_conseq = self._get_support(conseq, transactions)
                    supp_union = self._get_support(itemset, transactions)

                    conf = supp_union / supp_ante if supp_ante > 0 else 0
                    lift = supp_union / (supp_ante * supp_conseq) if supp_ante

                    if conf >= self.min_confidence and lift >= self.min_lift:
                        self.rules.append({
                            'antecedent': ante,
                            'consequent': conseq,
                            'confidence': conf,
                            'lift': lift
                        })

        self.rules.sort(key=lambda x: x['lift'], reverse=True)
        print(f"Правил: {len(self.rules)}")

    def recommend(self, basket, top_n=5):
        basket_set = set(basket)
        recs = {}
        for rule in self.rules:
            if set(rule['antecedent']).issubset(basket_set):
                for item in rule['consequent']:
                    if item not in basket_set:
                        if item not in recs or rule['lift'] > recs[item]['lift
                            recs[item] = {
                                'item': item,
                                'confidence': rule['confidence'],
                                'lift': rule['lift']
                            }
        result = sorted(recs.values(), key=lambda x: x['lift'], reverse=True)
        return result[:top_n]
```

In [ ]:

```
MIN_SUPPORT = 0.05
MIN_CONFIDENCE = 0.3
MIN_LIFT = 1.0

print("\nAPRIORI:")
start = time.time()
apriori = Apriori(MIN_SUPPORT, MIN_CONFIDENCE, MIN_LIFT)
apriori.fit(transactions_sample)
apriori_time = time.time() - start
print(f"Время: {apriori_time:.2f} сек")

print("\nFP-GROWTH:")
start = time.time()
fpgrowth = FPGrowth(MIN_SUPPORT, MIN_CONFIDENCE, MIN_LIFT)
fpgrowth.fit(transactions_sample)
fpgrowth_time = time.time() - start
print(f"Время: {fpgrowth_time:.2f} сек")


print("Сравнение")
```

```python
print(f"Apriori:    {apriori_time:.2f} сек, правил: {len(apriori.rules)}")
print(f"FP-Growth:  {fpgrowth_time:.2f} сек, правил: {len(fpgrowth.rules)}")
print(f"FP-Growth быстрее в {apriori_time/fpgrowth_time:.2f} раз")


print("Проверка рекомендаций")

test_basket = ['WHITE HANGING HEART T-LIGHT HOLDER', 'REGENCY CAKESTAND 3 TIER
print(f"\nКорзина: {test_basket}")

print("\nРекомендации (FP-Growth):")
recommendations = fpgrowth.recommend(test_basket, top_n=5)
for i, rec in enumerate(recommendations, 1):
    print(f"  {i}. {rec['item']} (lift={rec['lift']:.2f}, уверенность={rec['co
```

```
APRIORI:
Правил: 10
Время: 0.21 сек

FP-GROWTH:
Правил: 16
Время: 0.07 сек
Сравнение
Apriori:    0.21 сек, правил: 10
FP-Growth:  0.07 сек, правил: 16
FP-Growth быстрее в 2.99 раз
Проверка рекомендаций

Корзина: ['WHITE HANGING HEART T-LIGHT HOLDER', 'REGENCY CAKESTAND 3 TIER']

Рекомендации (FP-Growth):
  1. KNITTED UNION FLAG HOT WATER BOTTLE (lift=3.74, уверенность=32.89%)
  2. RED WOOLLY HOTTIE WHITE HEART. (lift=3.11, уверенность=34.21%)
```